

2025 – Lab Exam 03 Report

Student ID	IT23224384
Batch	Y2.S2.WE.IT.03.02
Marking Guide	
1. Functionality: How well the core and bonus features are implemented	4
2. Creativity & Usability: Clean and intuitive UI/UX design	2
3. Validation & Error Handling: Proper input validation, error handling, and user feedback implementation	2
4. Data Persistence: Proper use of SharedPreferences and Internal Storage	2
Total Marks	10
Evaluator	

Description:

Application Name

FINORA – TRACK YOUR EXPENSES

Overview

Finora is an innovative and user-friendly personal finance management application tailored for individuals who wish to gain control over their finances without the intricacies associated with conventional budgeting methods. Suitable for students, professionals, entrepreneurs, and homemakers alike, Finora enables users to oversee their income, monitor expenditures, establish savings objectives, and acquire insights into their financial behaviors—accessible at any time and from any location. The application ensures a smooth user experience by providing personalized budgeting strategies, real-time expense monitoring, financial goal establishment, and intelligent insights tailored to individual preferences. Finora removes the necessity for

spreadsheets or complicated tools, allowing users to enhance their savings, curb excessive spending, and maintain financial oversight effortlessly. With functionalities such as visual representations of expenses, customized budget notifications, and monthly financial summaries, users can cultivate better financial practices conveniently through their mobile devices. Finora delivers financial transparency and autonomy directly to your hands.

First Time Using the App

New users can sign up and log in to unlock the full potential of **Finora**. Upon logging in, users will be prompted to enter their income sources, recurring expenses, and savings goals to generate a tailored financial plan.

Using the App

After account creation and login, users are directed to the **dashboard**, where they can explore spending trends, track budgets, and monitor their progress. Users can log expenses, categorize transactions, and receive spending alerts or reminders.

Finora **enables users to:**

- ✓ Track daily, weekly, and monthly expenses with ease
- ✓ Categorize income and spending (e.g., food, bills, transport)
- ✓ Set personalized savings goals and monitor progress
- ✓ View reports and charts for better financial decision-making
- ✓ Stay motivated through savings milestones and challenges

Finora offers a clear, flexible path to financial wellness, allowing users to manage their money and meet their goals without stress.

Problem Statement

In today's fast-paced world, managing personal finances is often overlooked. Many individuals struggle with tracking expenses, overspending, and achieving savings goals due to the lack of time, knowledge, or access to effective tools. Traditional methods like notebooks or spreadsheets are inefficient and tedious, while many finance apps lack simplicity or personalization.

Finora addresses this gap by providing a modern, user-friendly solution for everyday money management. By offering personalized budgets, visual analytics, flexible tracking, and smart reminders, Finora makes financial health easy, engaging, and achievable. It empowers users to build better financial habits, gain clarity, and move toward their money goals confidently.

Target Audience

Finora is tailored to support a wide range of users who seek financial clarity and simplicity:

- ✓ **Busy Professionals** – Individuals managing tight schedules who need a quick, efficient way to handle budgets and expenses.
- ✓ **Students** – Young adults aiming to track allowances, part-time income, and spending without complex tools.
- ✓ **Stay-at-Home Parents** – Parents who want to manage household budgets and stay on top of family expenses.
- ✓ **Freelancers & Entrepreneurs** – Independent earners looking to categorize income, monitor business spending, and plan savings.
- ✓ **Budgeting Beginners** – Anyone new to money management needing a friendly, guided finance tracker.

Key Features

• Smart Budgeting (Main Function)

- Create customized budgets based on income, spending habits, and savings goals.
- Personalized spending insights to keep you on track.
- Easy-to-understand visual breakdowns of your finances.

• Expense & Income Tracker

- Quickly log transactions and categorize them (food, bills, travel, etc.).
- Add notes or tags for detailed tracking.
- Monitor recurring expenses and alerts for bill payments.

• Goal-Oriented Saving Plans

- Set and track financial goals like emergency funds, vacation savings, or debt repayment.
- See real-time progress and adjust contributions easily.
- Encouraging milestones to help you stay motivated.

• Financial Insights & Recommendations

- Smart insights based on your habits and budget behavior.
- Suggested adjustments to reduce overspending or improve savings.
- Weekly reports for better decision-making.

• Reminders & Alerts

- Bill due reminders, low-balance alerts, and custom spending limit warnings.
- Daily and monthly summaries to keep you updated.

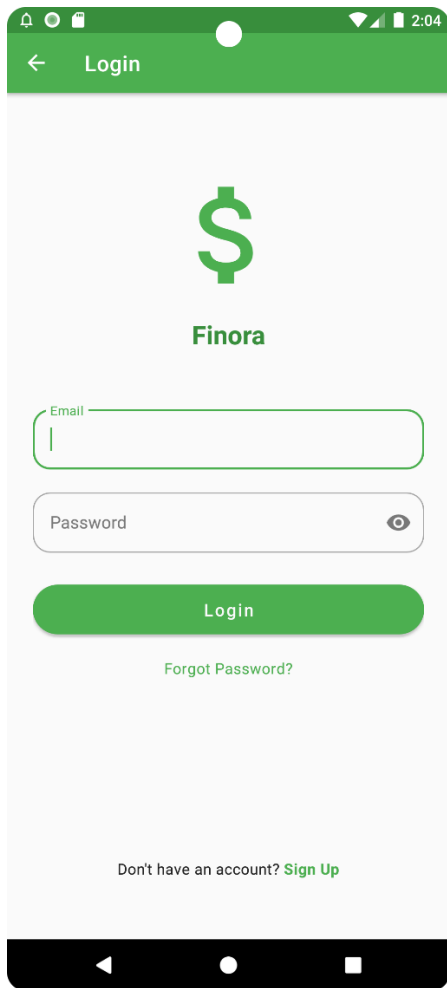
- **Data Sync & Security**

Sync across devices and cloud backup.

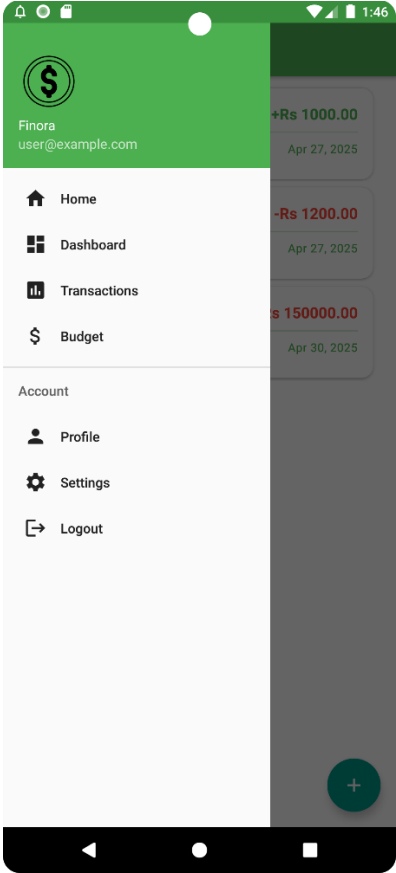
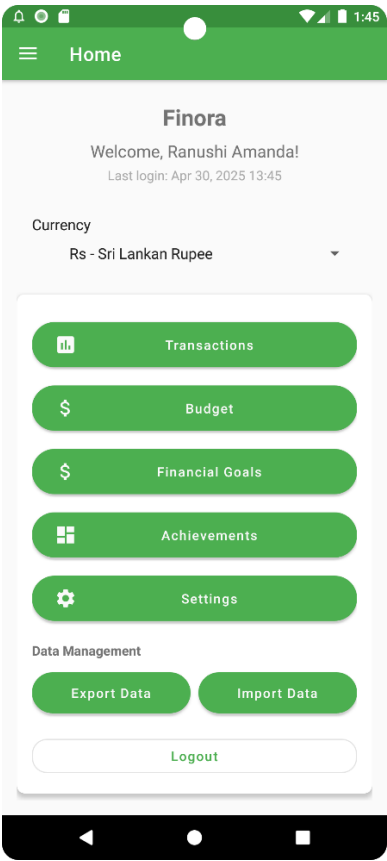
Your financial data stays private with bank-grade encryption and secure local storage.

Screenshots:

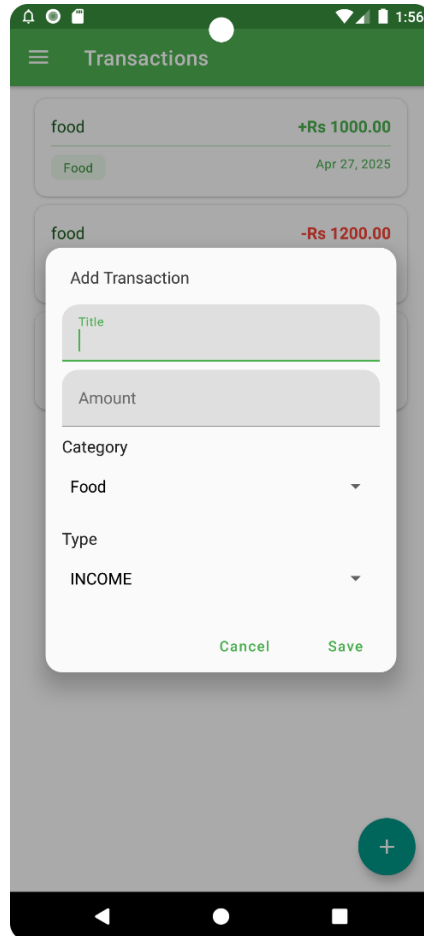
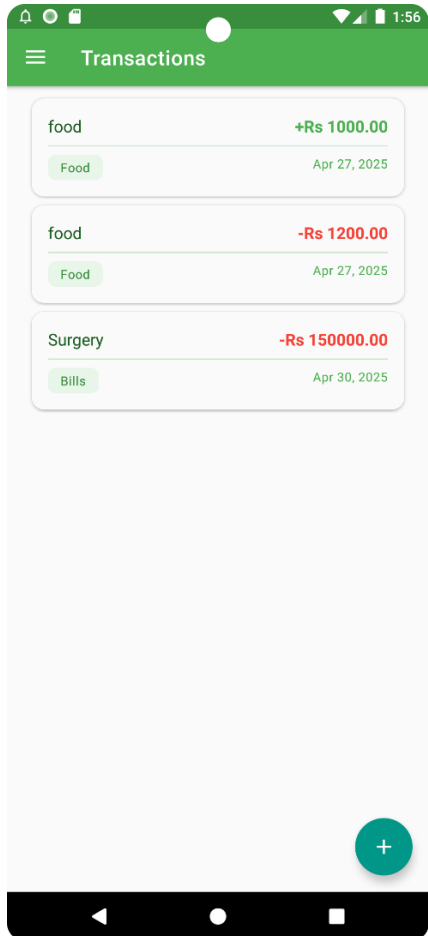
Login Page:



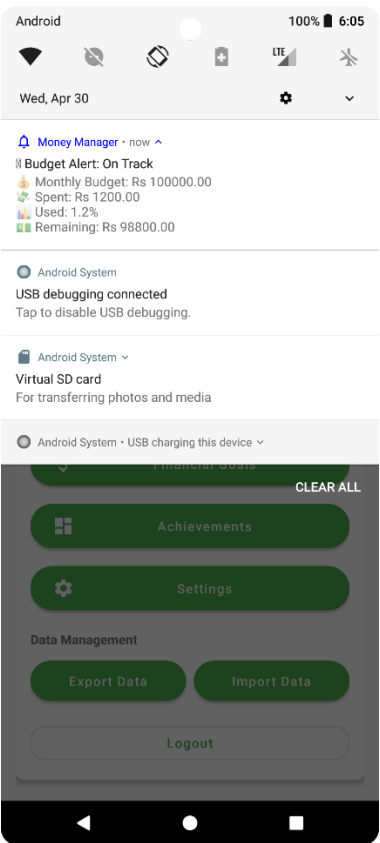
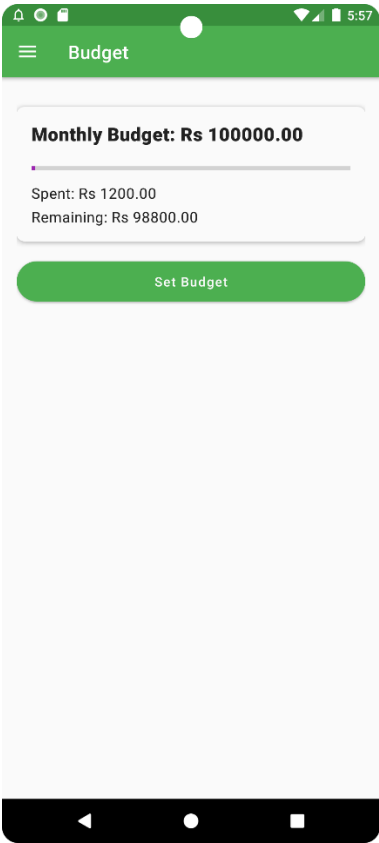
Home Screen:



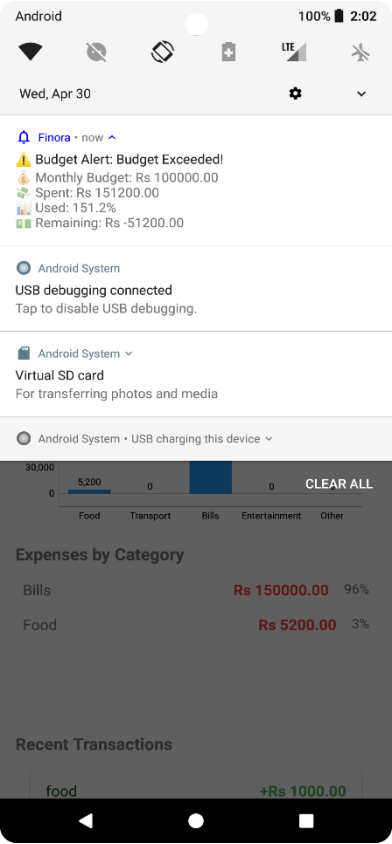
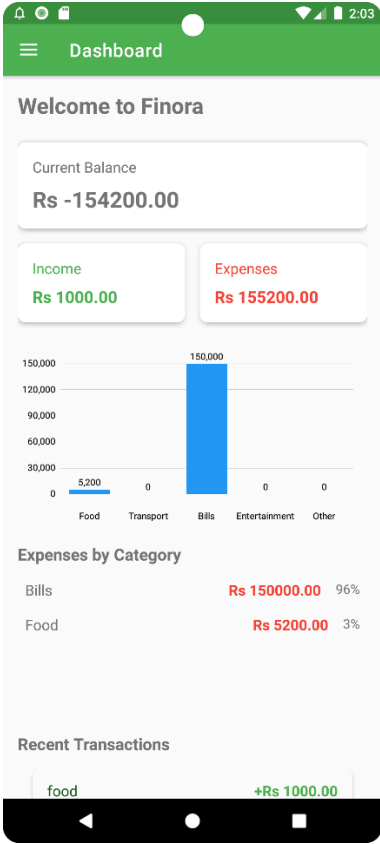
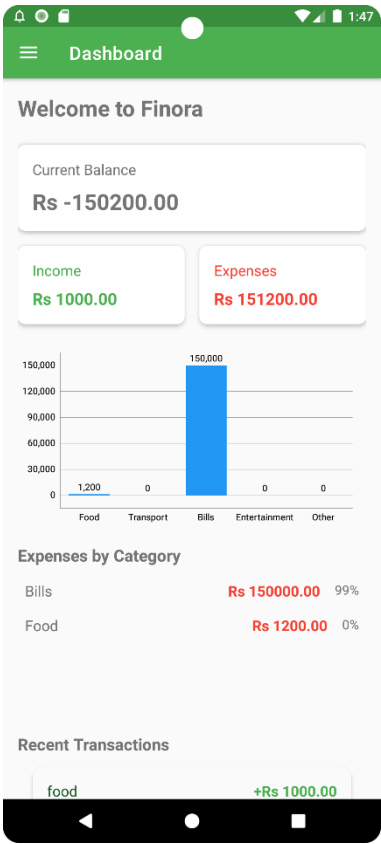
Transactions:



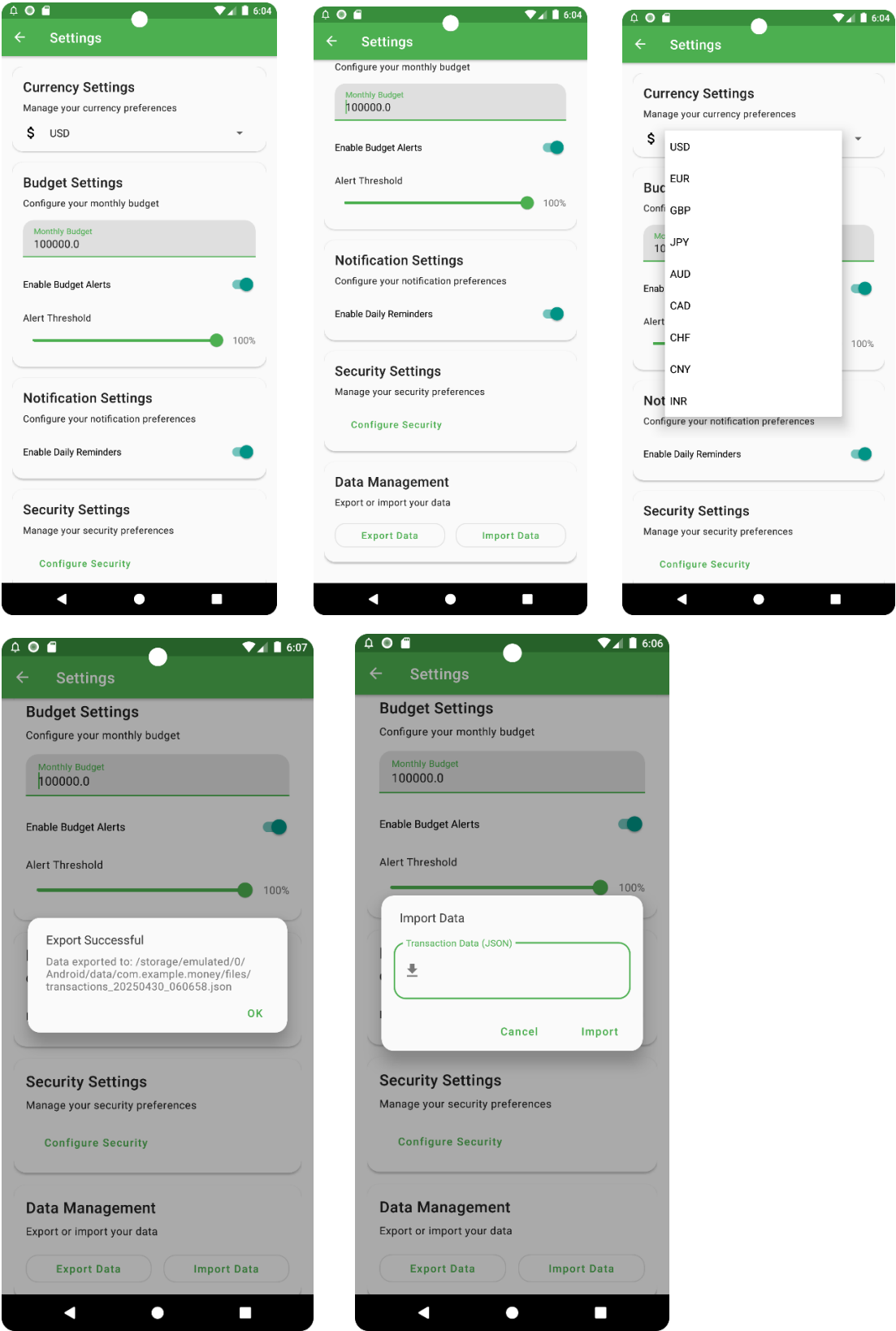
Budget Set:

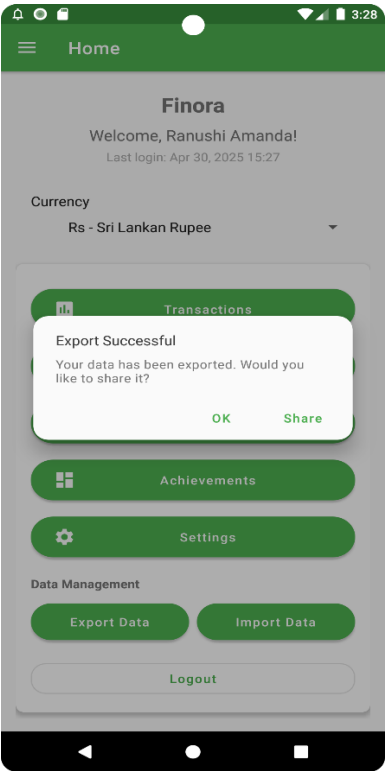
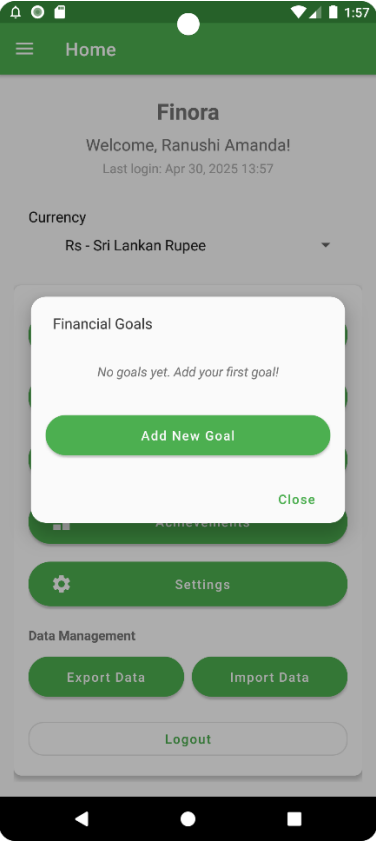
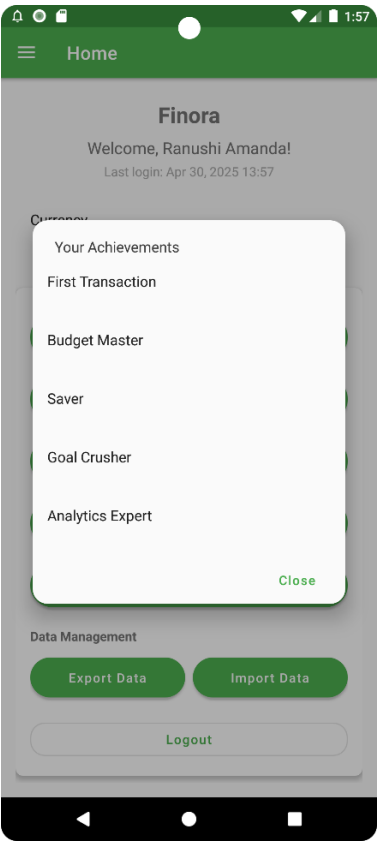
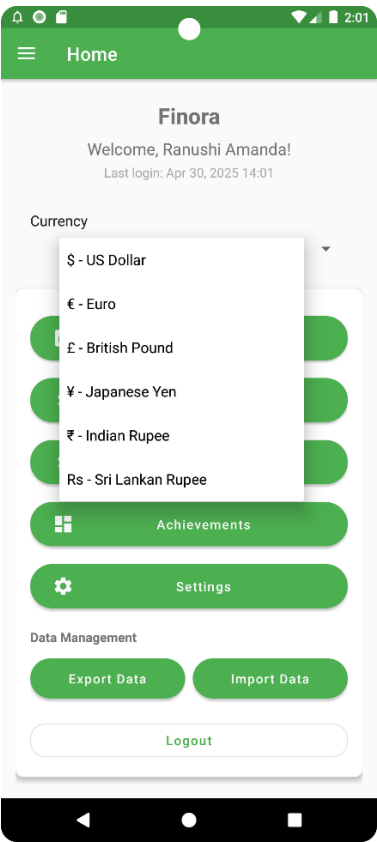


Dashboard:

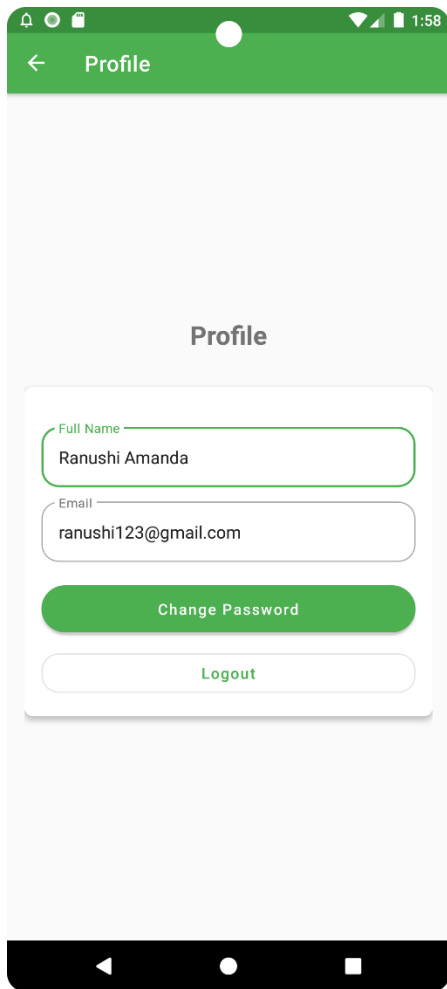


Settings:





Profile:



A mobile application interface for a user profile. At the top is a green header bar with a back arrow, the title 'Profile', and status icons (notification, signal, battery, time 1:58). Below the header is a light gray background with the title 'Profile' centered. A white card contains the following elements: a 'Full Name' label above a text input with 'Ranushi Amanda'; an 'Email' label above a text input with 'ranushi123@gmail.com'; a green 'Change Password' button; and a 'Logout' button with green text. The bottom of the screen shows a black Android navigation bar.

Profile

Full Name

Ranushi Amanda

Email

ranushi123@gmail.com

Change Password

Logout

Content of xml files of Strings and Colors:

Strings

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Finora</string>
    <string name="navigation_drawer_open">Open navigation drawer</string>
    <string name="navigation_drawer_close">Close navigation drawer</string>
    <string name="nav_header_desc">Navigation header</string>
    <string name="change_password">Change Password</string>
    <string name="email">Email</string>
    <string name="full_name">Full Name</string>
    <string name="profile">Profile</string>
    <string name="logout">Logout</string>
    <string name="currency_settings">Currency Settings</string>
    <string name="manage_your_currency_preferences">Manage your currency preferences</string>
    <string name="budget_settings">Budget Settings</string>
    <string name="configure_your_monthly_budget">Configure your monthly budget</string>
    <string name="monthly_budget">Monthly Budget</string>
    <string name="enable_budget_alerts">Enable Budget Alerts</string>
    <string name="alert_threshold">Alert Threshold</string>
    <string name=".80">80%</string>
    <string name="notification_settings">Notification Settings</string>
    <string name="configure_your_notification_preferences">Configure your notification preferences</string>
    <string name="enable_daily_reminders">Enable Daily Reminders</string>
    <string name="security_settings">Security Settings</string>
    <string name="manage_your_security_preferences">Manage your security preferences</string>
    <string name="configure_security">Configure Security</string>
    <string name="data_management">Data Management</string>
```

```
<resources>
    <string name="currency_settings">Currency Settings</string>
    <string name="manage_your_currency_preferences">Manage your currency preferences</string>
    <string name="budget_settings">Budget Settings</string>
    <string name="configure_your_monthly_budget">Configure your monthly budget</string>
    <string name="monthly_budget">Monthly Budget</string>
    <string name="enable_budget_alerts">Enable Budget Alerts</string>
    <string name="alert_threshold">Alert Threshold</string>
    <string name=".80">80%</string>
    <string name="notification_settings">Notification Settings</string>
    <string name="configure_your_notification_preferences">Configure your notification preferences</string>
    <string name="enable_daily_reminders">Enable Daily Reminders</string>
    <string name="security_settings">Security Settings</string>
    <string name="manage_your_security_preferences">Manage your security preferences</string>
    <string name="configure_security">Configure Security</string>
    <string name="data_management">Data Management</string>
    <string name="export_or_import_your_data">Export or import your data</string>
    <string name="export_data">Export Data</string>
    <string name="import_data">Import Data</string>
    <string name="add_transaction">Add transaction</string>
</resources>
```

Colors

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3     <!-- Primary colors - -->
4     <color name="primary_50">#E8F5E9</color>
5     <color name="primary_100">#C8E6C9</color>
6     <color name="primary_200">#A5D6A7</color>
7     <color name="primary_300">#81C784</color>
8     <color name="primary_400">#66BB6A</color>
9     <color name="primary_500">#4CAF50</color>
10    <color name="primary_600">#43A047</color>
11    <color name="primary_700">#388E3C</color>
12    <color name="primary_800">#2E7D32</color>
13    <color name="primary_900">#1B5E20</color>
14
15    <!-- Secondary colors - -->
16    <color name="secondary_50">#E0F2F1</color>
17    <color name="secondary_100">#B2DFDB</color>
18    <color name="secondary_200">#80CBC4</color>
19    <color name="secondary_300">#4DB6AC</color>
20    <color name="secondary_400">#26A69A</color>
21    <color name="secondary_500">#009688</color>
22    <color name="secondary_600">#00897B</color>
23    <color name="secondary_700">#00796B</color>
24    <color name="secondary_800">#00695C</color>
25    <color name="secondary_900">#004D40</color>
26
```

```
27 <!-- Accent colors - Coral -->
28 <color name="accent_50">#FFEBEE</color>
29 <color name="accent_100">#FFCDD2</color>
30 <color name="accent_200">#EF9A9A</color>
31 <color name="accent_300">#E57373</color>
32 <color name="accent_400">#EF5350</color>
33 <color name="accent_500">#F44336</color>
34 <color name="accent_600">#E53935</color>
35 <color name="accent_700">#D32F2F</color>
36 <color name="accent_800">#C62828</color>
37 <color name="accent_900">#B71C1C</color>
38
39 <!-- Neutral colors - Modern Gray -->
40 <color name="neutral_50">#FAFAFA</color>
41 <color name="neutral_100">#F5F5F5</color>
42 <color name="neutral_200">#EEEEEE</color>
43 <color name="neutral_300">#E0E0E0</color>
44 <color name="neutral_400">#BDBDBD</color>
45 <color name="neutral_500">#9E9E9E</color>
46 <color name="neutral_600">#757575</color>
47 <color name="neutral_700">#616161</color>
48 <color name="neutral_800">#424242</color>
49 <color name="neutral_900">#212121</color>
50
51 <!-- Semantic colors -->
```

Money > app > src > main > res > values > colors.xml

1:39 LF UTF-8 4 spaces

```
colors.xml
2 <resources>
47 <color name="neutral_700">#616161</color>
48 <color name="neutral_800">#424242</color>
49 <color name="neutral_900">#212121</color>
50
51 <!-- Semantic colors -->
52 <color name="success">#4CAF50</color>
53 <color name="error">#F44336</color>
54 <color name="warning">#FF9800</color>
55 <color name="info">#2196F3</color>
56
57 <!-- Legacy colors (for backward compatibility) -->
58 <color name="purple_200">@color/primary_200</color>
59 <color name="purple_500">@color/primary_500</color>
60 <color name="purple_700">@color/primary_700</color>
61 <color name="teal_200">@color/secondary_200</color>
62 <color name="teal_700">@color/secondary_700</color>
63 <color name="black">@color/neutral_900</color>
64 <color name="white">@color/neutral_50</color>
65 <color name="colorPrimary">@color/primary_500</color>
66 </resources>
```

Money > app > src > main > res > values > colors.xml 1:39 LF UTF-8 4 spaces

User Preferences:

```
UserPreferencesManager.java
1 package com.example.money.managers;
2
3 import ...
13
14 public class UserPreferencesManager { 12 usages
15     private static final String PREF_NAME = "user_preferences"; 1 usage
16     private static final String KEY_PASSWORD = "password"; 5 usages
17     private static final String KEY_USERNAME = "username"; 4 usages
18     private static final String KEY_IS_LOGGED_IN = "is_logged_in"; 5 usages
19     private static final String KEY_HAS_PASSWORD = "has_password"; 4 usages
20
21     private static UserPreferencesManager instance; 3 usages
22     private final SharedPreferences sharedPreferences; 13 usages
23     private final com.example.money.data.UserManager kotlinUserManager; 1 usage
24
25     private UserPreferencesManager(Context context) { 1 usage
26         sharedPreferences = context.getApplicationContext()
27             .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
28         kotlinUserManager = new com.example.money.data.UserManager(context);
29     }
30
31     public static synchronized UserPreferencesManager getInstance(Context context) {
32         if (instance == null) {
33             instance = new UserPreferencesManager(context);
34         }
35         return instance;
36     }
37 }
```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces

```

14  public class UserPreferencesManager { 12 usages
36  }
37
38  public boolean register(String username, String password) { no usages
39      if (username == null || password == null || username.isEmpty() || password.isEmpty()) {
40          return false;
41      }
42
43      // Check if username already exists
44      if (sharedPreferences.contains(KEY_USERNAME)) {
45          return false;
46      }
47
48      // Hash the password
49      String hashedPassword = hashPassword(password);
50      if (hashedPassword == null) {
51          return false;
52      }
53
54      // Save user credentials
55      SharedPreferences.Editor editor = sharedPreferences.edit();
56      editor.putString(KEY_USERNAME, username);
57      editor.putString(KEY_PASSWORD, hashedPassword);
58      editor.putBoolean(KEY_IS_LOGGED_IN, true);
59      editor.putBoolean(KEY_HAS_PASSWORD, true);
60      return editor.commit();

```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces

```

14  public class UserPreferencesManager { 12 usages
38  public boolean register(String username, String password) { no usages
59      editor.putBoolean(KEY_HAS_PASSWORD, true);
60      return editor.commit();
61  }
62
63  public boolean login(String username, String password) {
64      if (username == null || password == null || username.isEmpty() || password.isEmpty()) {
65          return false;
66      }
67
68      // Get stored username and password
69      String storedUsername = sharedPreferences.getString(KEY_USERNAME, defValue: "");
70      String storedPassword = sharedPreferences.getString(KEY_PASSWORD, defValue: "");
71
72      // Hash the provided password
73      String hashedPassword = hashPassword(password);
74      if (hashedPassword == null) {
75          return false;
76      }
77
78      // Check if credentials match
79      if (username.equals(storedUsername) && hashedPassword.equals(storedPassword)) {
80          // Set logged in status
81          SharedPreferences.Editor editor = sharedPreferences.edit();
82          editor.putBoolean(KEY_IS_LOGGED_IN, true);

```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces

```
14 public class UserPreferencesManager { 12 usages
63 public boolean login(String username, String password) {
79     if (username.equals(storedUsername) && hashedPassword.equals(storedPassword)) {
80         // Set logged in status
81         SharedPreferences.Editor editor = sharedPreferences.edit();
82         editor.putBoolean(KEY_IS_LOGGED_IN, true);
83         editor.apply();
84         return true;
85     }
86
87     return false;
88 }
89
90 public void logout() { 1 usage
91     SharedPreferences.Editor editor = sharedPreferences.edit();
92     editor.putBoolean(KEY_IS_LOGGED_IN, false);
93     editor.apply();
94 }
95
96 > public boolean isLoggedIn() { return sharedPreferences.getBoolean(KEY_IS_LOGGED_IN, defValue: false); }
99
100 > public String getUsername() { return sharedPreferences.getString(KEY_USERNAME, defValue: ""); }
103
104 > public boolean hasPassword() { return sharedPreferences.getBoolean(KEY_HAS_PASSWORD, defValue: false); }
107
108 public boolean verifyPassword(String password) { 1 usage
```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces

```
14 public class UserPreferencesManager { 12 usages
100 > public String getUsername() { return sharedPreferences.getString(KEY_USERNAME, defValue: ""); }
103
104 > public boolean hasPassword() { return sharedPreferences.getBoolean(KEY_HAS_PASSWORD, defValue: false); }
107
108 public boolean verifyPassword(String password) { 1 usage
109     if (password == null || password.isEmpty()) {
110         return false;
111     }
112
113     String storedPassword = sharedPreferences.getString(KEY_PASSWORD, defValue: "");
114     String hashedPassword = hashPassword(password);
115
116     return hashedPassword != null && hashedPassword.equals(storedPassword);
117 }
118
119 public boolean changePassword(String newPassword) { 1 usage
120     if (newPassword == null || newPassword.isEmpty()) {
121         return false;
122     }
123
124     String hashedPassword = hashPassword(newPassword);
125     if (hashedPassword == null) {
126         return false;
127     }
128 }
```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces


```
14 public class UserPreferencesManager { 12 usages
119 public boolean changePassword(String newPassword) { 1 usage
129     SharedPreferences.Editor editor = sharedPreferences.edit();
130     editor.putString(KEY_PASSWORD, hashedPassword);
131     editor.putBoolean(KEY_HAS_PASSWORD, true);
132     return editor.commit();
133 }
134
135 public boolean setNewPassword(String newPassword) { 1 usage
136     if (newPassword == null || newPassword.isEmpty()) {
137         return false;
138     }
139
140     String hashedPassword = hashPassword(newPassword);
141     if (hashedPassword == null) {
142         return false;
143     }
144
145     SharedPreferences.Editor editor = sharedPreferences.edit();
146     editor.putString(KEY_PASSWORD, hashedPassword);
147     editor.putBoolean(KEY_HAS_PASSWORD, true);
148     editor.putBoolean(KEY_IS_LOGGED_IN, true);
149     return editor.commit();
150 }
151
152 @ private String hashPassword(String password) { 5 usages
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
        return Base64.encodeToString(hash, Base64.NO_WRAP);
    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
        return null;
    }
}
```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces

```
14 public class UserPreferencesManager { 12 usages
135 public boolean setNewPassword(String newPassword) { 1 usage
144     SharedPreferences.Editor editor = sharedPreferences.edit();
145     editor.putString(KEY_PASSWORD, hashedPassword);
146     editor.putBoolean(KEY_HAS_PASSWORD, true);
147     editor.putBoolean(KEY_IS_LOGGED_IN, true);
148     return editor.commit();
149 }
150
151
152 @ private String hashPassword(String password) { 5 usages
153     try {
154         MessageDigest digest = MessageDigest.getInstance("SHA-256");
155         byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
156         return Base64.encodeToString(hash, Base64.NO_WRAP);
157     } catch (NoSuchAlgorithmException e) {
158         e.printStackTrace();
159         return null;
160     }
161 }
162 }
```

Money > app > src > main > java > com > example > money > managers > UserPreferencesManager > register 50:1 CRLF UTF-8 4 spaces